Assignment -2

* Problem statement
  Write C program for sparse matrix realization and operation on it - Transpose, Fast Transpose

* Objective
  1. To study the concept of Sparse matrix, how it is stored & displayed.
  2. To understand the implementation of sparse matrix operations - Simple Transpose, Fast Transpose.

* Implementation

- Pseudo code:
  write pseudo code for input matrix to compact form, Simple Transpose and fast Transpose

- Time Complexity:
  Find out the time complexity of Simple Transpose & Fast Transpose.

- Conclusion:
  Thus, implemented sparse matrix operations assignment. This system is able to perform different operation on sparse matrices such as simple & fast transpose and their time complexities.

- Platform :
  - 64 bit open source linux or its derivatives
  - open source C programming tool like gcc/ Eclipse editor.

* FAQ s

a. What is sparse matrix? List the applications?

→

A sparse matrix is a matrix in which most of the elements are 0.
In other words, the number of zero elements is much greater than the number of zero-non elements.

Applications:

- Graph Representation
- Scientific & Engineering Computations
- Image processing
- Machine learning
- Data Mining
- Computer Networks

b. Represent sparse matrices with suitable data structures & Explain with an example Simple & fast transpose?

→

Representation of SM:

- Triplet (compact) Rep. → most common

each non zero member is ~~represented~~ stored as (Row, Column, value)

First row.
(no. of rows, no. of columns, no. of non zero)

ex:
0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	6	2	0	0

Triplet Representation.

Row	Column	value
4	5	6
0	2	3
0	4	4
1	2	3
1	3	7
3	1	2
3	2	6

1st row
- 4 rows
- 5 columns
- 6 non zero elements

Remaining rows contains
(Row no., Column no., value)

Simple Transpose
convert rows into columns by scanning one - by - one column
① copy header row.
② For each row
- scan all none zero elements
- when column matches, interchange row, & column.

Date:

ex: · orginal

Row	column	value
4	5	6
0	2	3
0	4	4
1	2	5
1	3	7
3	1	2
3	2	6

S.T

Row	column	value
5	4	6
1	2	2
2	0	3
2	1	5
3	3	6
3	1	7
4	0	4

Time complexity

O(Columns X No.Non zero elements)

Fast Transpose

Improve efficiency by using more arrays.

- Total[ ] - store non zero element
- Index[ ] - store

(1) Copy the header

(2) Count non zero element using Total[ ]

(3) Compute starting position using Index[ ]

(4) Place every element directly into correct block

(5) Increment corresponding index.

ex: Total[ ]

column	0	1	2	3	4
count	0	1	3	1	1

Index[ ]

Column	0	1	2	3	4
Start Pos.	1	1	2	5	6

final

Row	column	value
5	4	6
1	3	2
2	0	3
2	1	5
2	3	6
3	1	7
4	0	1

Time complexity

O(columns + non zero elements)

3. Find out the addition of 2 sparse matrix in triplet form and also find simple & fast transpose?

→

M1

4	5	6
0	3	5
1	3	6
1	4	48
2	3	4
3	2	45
4	1	2

M2

4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	4	21

Addition

Position	M1	M2	Sum
(0,3)	5	7	12
(0,4)	0	6	6
(1,3)	8	0	8
(1,4)	45	4	49
(2,1)	0	8	8
(2,3)	4	0	4
(3,2)	45	45	90
(4,1)	2	0	2
(4,4)	0	21	21

Total non zero elements = 9

Result in triplet

Row	Column	Value
4	5	9
0	3	12
0	4	6
1	3	8
1	4	49
2	1	8
2	3	4
3	2	90
4	1	2
4	4	2

Simple Transpose

Row	Column	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

Row	Column	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

Total ( )

Column	0	1	2	3	4
Count	0	1	1	3	1

* Pseudo code
Input matrix to compact form
Start
Read rows, columns
k ← 1
for i = 0 to rows - 1
for j = 0 to columns - 1
Read matrix [i] [j]
if matrix [i] [j] to
compact [k] . row = i
compact [k] . col = j
compact [k] . value = matrix [i] [j]
k = k + 1
End if
End for
End for
Compact [0] . row = rows
compact [0] . col = columns
compact [0] . value = k - 1
Step.

Simple Transpose
Transpose. [0] . row = original column
Transpose [0] . col = original . rows
Transpose [0] . value = original . non-zero
k ← 1
for each column c
for each non-zero element
if element . column = c
Transpose [k] . row = element . column
Transpose [k] . col = element . row

```
        Transpose [k].value = element.value
        k = k+1
      End if
    End for
  End for
  Stop
```

Fast Transpose

```
Start
Store transpose header
For each non zero element
      Total [column] +
End for
Index [0] = 1
for i = 1 to column - 1
    Index [i] = Index [i-1] + total [i-1]
End for
For each non zero element
    position = Index [element.column]
    fast [position]. row = element column
    fast [position] col = element. row
    fast [position] .value = element value
    Index [element. column] + 1
End for
Stop
```

23/1/26

Time complexity	
operation	Time complexity
Compact representation	O(m x n)
Simple Transpose	O(col x non zero element)
Fast transpose	O(col + non zero element)

Code:

```
#include <stdio.h>
#define MAX 100
void compact(int A[MAX][MAX], int m, int n, int B[MAX][3]) {
   int k = 1;
   B[0][0] = m;
   B[0][1] = n;

   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
         if (A[i][j] != 0) {
            B[k][0] = i;
            B[k][1] = j;
            B[k][2] = A[i][j];
            k++;
         }
      }
   }
   B[0][2] = k - 1;
}

void simpleTranspose(int A[MAX][3], int B[MAX][3]) {
   int m = A[0][0];
   int n = A[0][1];
   int t = A[0][2];

   B[0][0] = n;
   B[0][1] = m;
   B[0][2] = t;

   if (t <= 0) return;

   int q = 1;
   for (int col = 0; col < n; col++) {
      for (int p = 1; p <= t; p++) {
         if (A[p][1] == col) {
            B[q][0] = A[p][1];
            B[q][1] = A[p][0];
            B[q][2] = A[p][2];
            q++;
         }
      }
   }
}
void fastTranspose(int A[MAX][3], int B[MAX][3]) {
   int m = A[0][0];
   int n = A[0][1];
```

```
    int t = A[0][2];

    int S[MAX], T[MAX];

    B[0][0] = n;
    B[0][1] = m;
    B[0][2] = t;

    if (t <= 0) return;

    for (int i = 0; i < n; i++) {
        S[i] = 0;
    }

    for (int i = 1; i <= t; i++) {
        S[A[i][1]] = S[A[i][1]] + 1;
    }

    T[0] = 1;
    for (int i = 1; i < n; i++) {
        T[i] = T[i - 1] + S[i - 1];
    }

    for (int i = 1; i <= t; i++) {
        int j = A[i][1];
        int pos = T[j];

        B[pos][0] = A[i][1];
        B[pos][1] = A[i][0];
        B[pos][2] = A[i][2];

        T[j]++;
    }
}

void displaySparse(int B[MAX][3]) {
    int t = B[0][2];
    printf("\nRow\tCol\tValue\n");
    printf("-----------------------\n");
    for (int i = 0; i <= t; i++) {
        printf("%d\t%d\t%d\n", B[i][0], B[i][1], B[i][2]);
    }
}

int main() {
    int A[MAX][MAX], C[MAX][3], B[MAX][3];
    int m, n, ch;
```

```
int compactCreated = 0;

do {
   printf("\n--- MENU ---");
   printf("\n1. Compact Form (Input Matrix)");
   printf("\n2. Simple Transpose");
   printf("\n3. Fast Transpose");
   printf("\n4. Exit");
   printf("\nEnter your choice: ");
   scanf("%d", &ch);

   switch (ch) {
      case 1:
         printf("Enter nos of rows (m) and columns (n): ");
         scanf("%d %d", &m, &n);

         printf("Enter matrix elements of A:\n");
         for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
               scanf("%d", &A[i][j]);
            }
         }

         compact(A, m, n, C);
         compactCreated = 1;
         printf("\nCompact form created in matrix C:");
         displaySparse(C);
         break;

      case 2:
         if (!compactCreated) {
            printf("\nPlease input matrix (Option 1) first!\n");
         } else {
            simpleTranspose(C, B);
            printf("\nSimple Transpose complete in matrix B:");
            displaySparse(B);
         }
         break;

      case 3:
         if (!compactCreated) {
            printf("\nPlease input matrix (Option 1) first!\n");
         } else {
            fastTranspose(C, B);
            printf("\nFast Transpose complete in matrix B:");
            displaySparse(B);
         }
```

```
            break;

        case 4:
            printf("\nExiting program.\n");
            break;

        default:
            printf("\nInvalid choice!\n");
      }
   } while (ch != 4);

   return 0;
}
```

Input and Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:1

Enter nos of rows (m) and columns (n):4 4

Enter matrix elements of A:
1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1


Compact form created in matrix C:
Row     Col     Value
------------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1
```

```
Enter your choice:2


Simple Transpose complete in matrix B:
Row     Col     Value
------------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:3


Fast Transpose complete in matrix B:
Row     Col     Value
------------------------
4       4       4
0       0       1
1       1       1
2       2       1
3       3       1
```

```
--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:5


Invalid choice!

--- MENU ---
1. Compact Form (Input Matrix)
2. Simple Transpose
3. Fast Transpose
4. Exit
Enter your choice:4


Exiting program.

Process finished with exit code 0
```